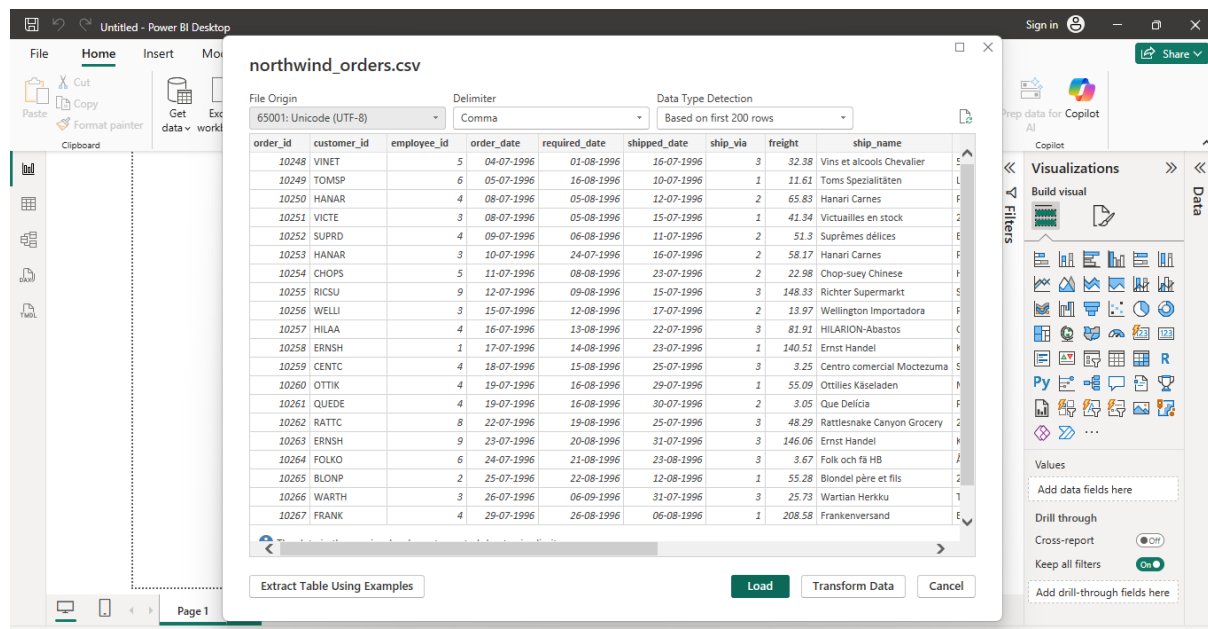
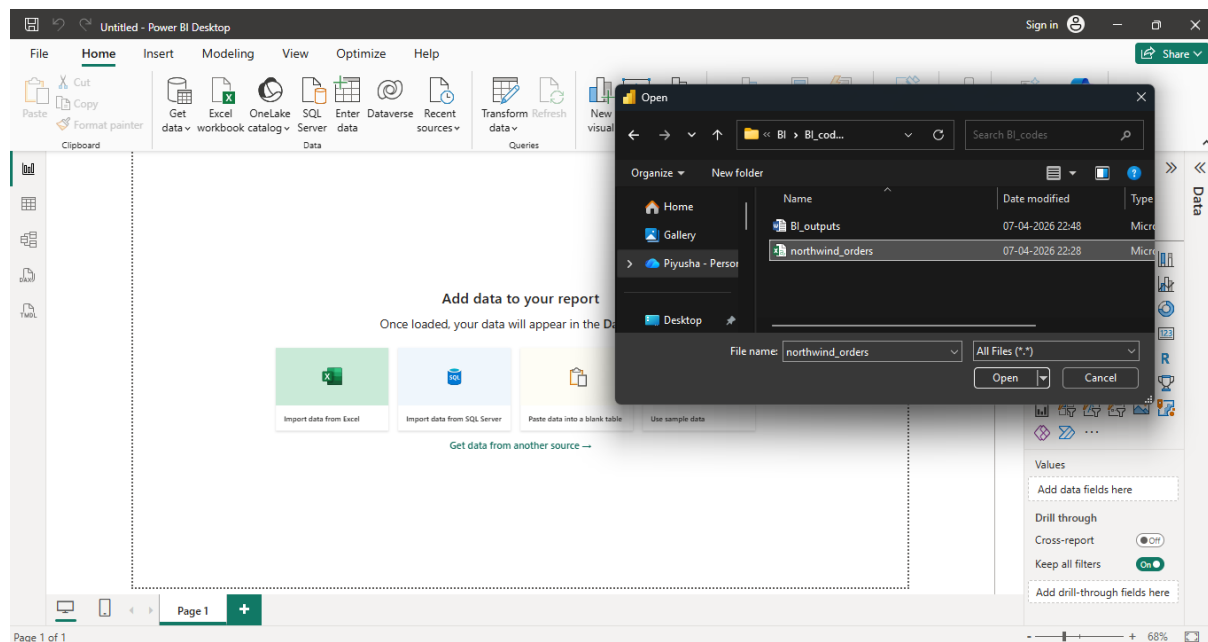
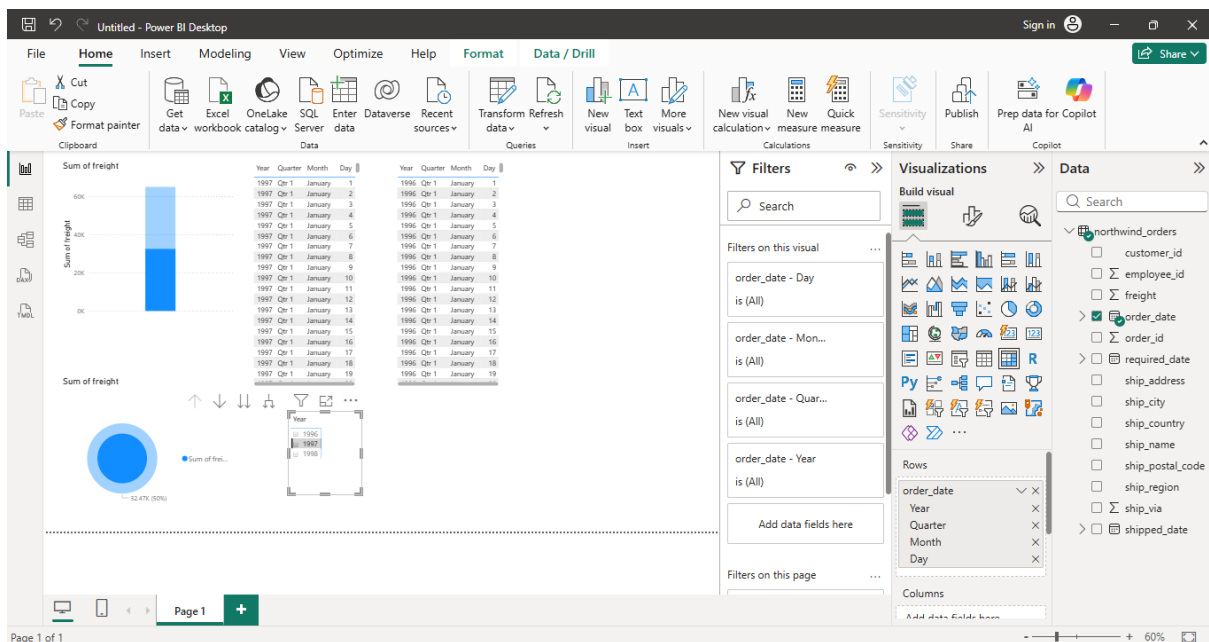
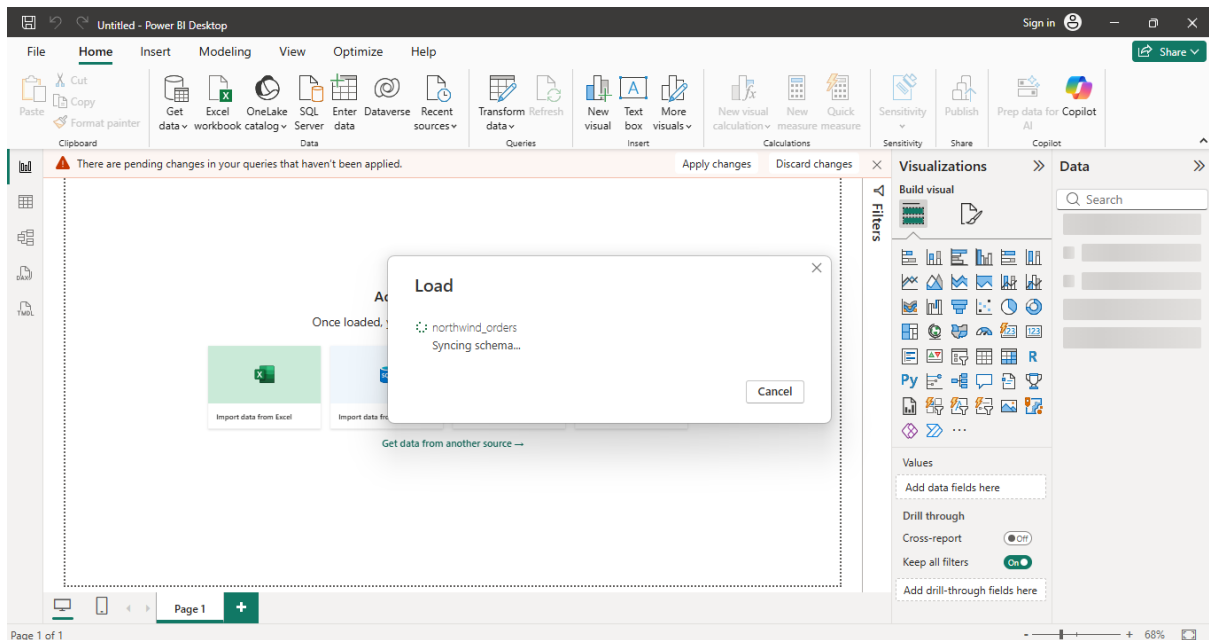


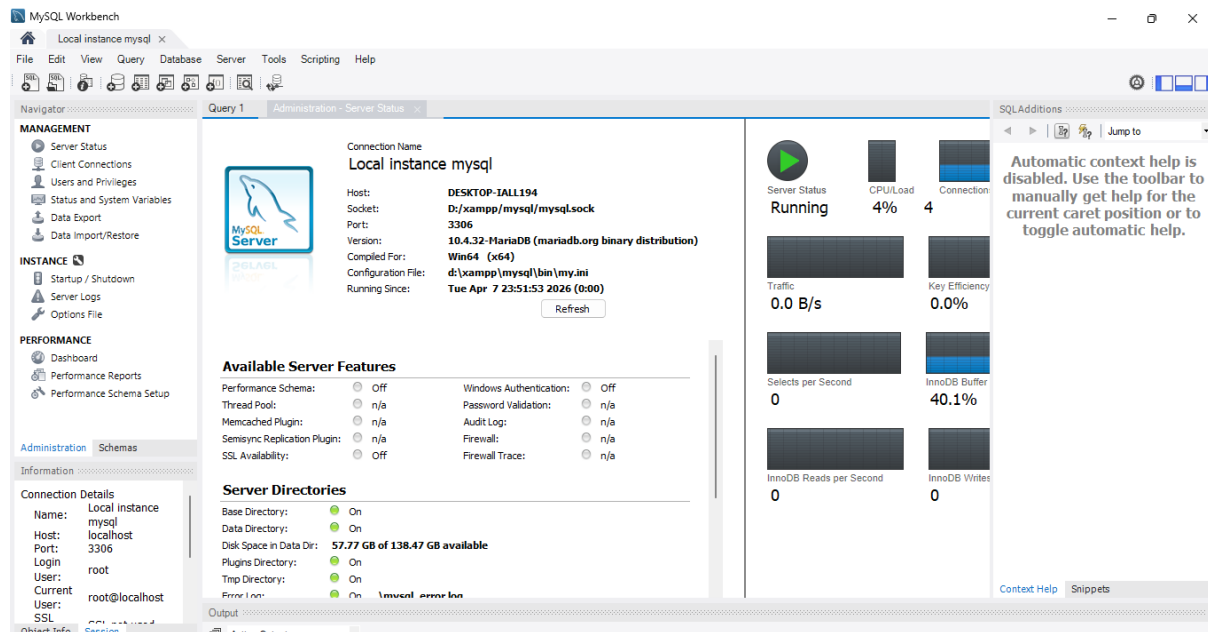
Practical 1: Import the legacy data from different sources such as (Excel , Sql Server, Oracle etc.) and load in the target system. (You can download sample database such as Adventure works, Northwind, foodmart etc.)



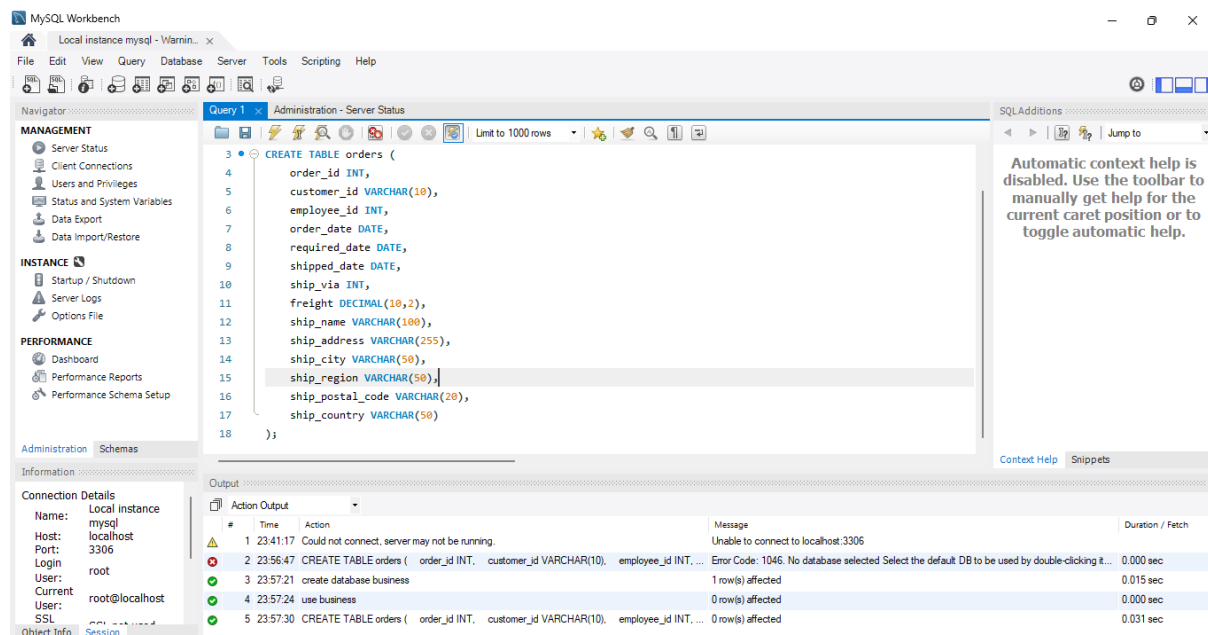


Practical 2: Perform the Extraction Transformation and Loading (ETL) process to construct the database in the Sql server.

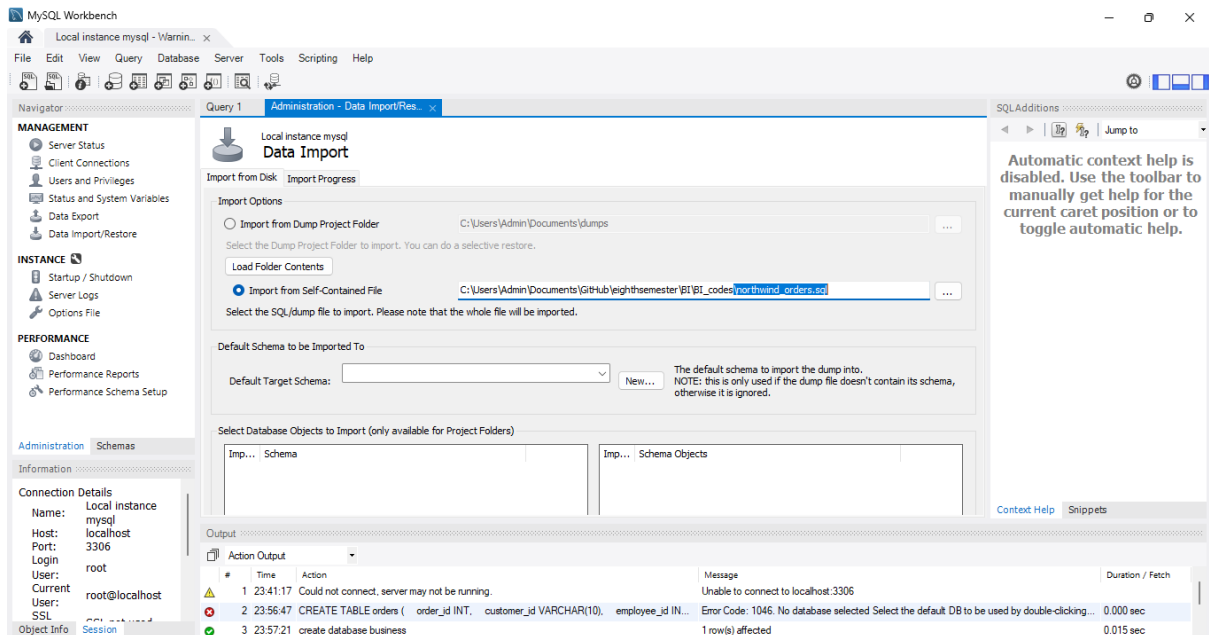
Open Mysql workbench



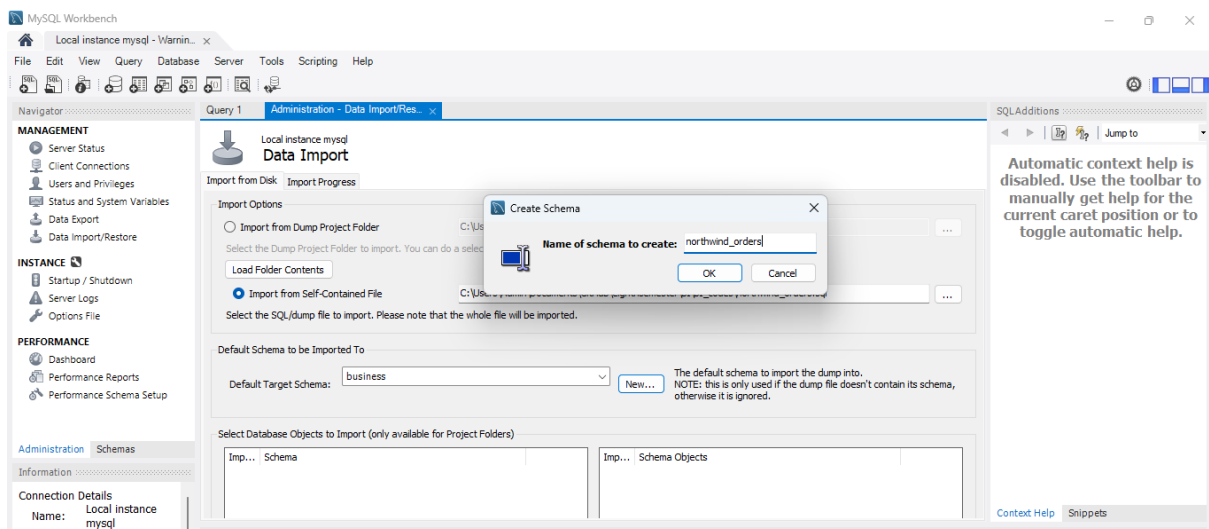
Then make a table to import our csv file, here I am using Northwind file



Then import the sql version of your CSV file into the workbench through data import



Create a new schema for the import



MySQL Workbench interface showing a query window with the following SQL statement:

```

7  ,order_date DATE NOT NULL
8  ,required_date DATE NOT NULL
9  ,shipped_date DATE
10 ,ship_via INTEGER NOT NULL
11 ,freight NUMERIC(7,2) NOT NULL
12 ,ship_name VARCHAR(34) NOT NULL
13 ,ship_address VARCHAR(46) NOT NULL
14 ,ship_city VARCHAR(15) NOT NULL
15 ,ship_region VARCHAR(13)
16 ,ship_postal_code VARCHAR(9)
17 ,ship_country VARCHAR(11) NOT NULL
18 );
19 • INSERT INTO mytable(order_id,customer_id,employee_id,order_date,required_date,shipped_date,ship_via,freight,ship_n
20 • INSERT INTO mytable(order_id,customer_id,employee_id,order_date,required_date,shipped_date,ship_via,freight,ship_n
21 • INSERT INTO mytable(order_id,customer_id,employee_id,order_date,required_date,shipped_date,ship_via,freight,ship_n
22 • INSERT INTO mytable(order_id,customer_id,employee_id,order_date,required_date,shipped_date,ship_via,freight,ship_n
23 • INSERT INTO mytable(order_id,customer_id,employee_id,order_date,required_date,shipped_date,ship_via,freight,ship_n
24 • INSERT INTO mytable(order_id,customer_id,employee_id,order_date,required_date,shipped_date,ship_via,freight,ship_n

```

The Output pane shows the following message:

```

836 00:12:33 INSERT INTO mytable(order_id,customer_id,employee_id,order_date,required_date,shipped_date,ship_via,freight,ship_n
1 row(s) affected, 2 warning(s): 1265 Data truncated for column 'order_date' at row 1 1265 D... 0.015 sec
837 00:12:33 INSERT INTO mytable(order_id,customer_id,employee_id,order_date,required_date,shipped_date,ship_via,freight,ship_n
1 row(s) affected, 3 warning(s): 1265 Data truncated for column 'order_date' at row 1 1265 D... 0.000 sec
838 00:12:33 INSERT INTO mytable(order_id,customer_id,employee_id,order_date,required_date,shipped_date,ship_via,freight,ship_n
1 row(s) affected, 2 warning(s): 1265 Data truncated for column 'order_date' at row 1 1265 D... 0.032 sec
839 00:12:33 INSERT INTO mytable(order_id,customer_id,employee_id,order_date,required_date,shipped_date,ship_via,freight,ship_n
1 row(s) affected, 2 warning(s): 1265 Data truncated for column 'order_date' at row 1 1265 D... 0.015 sec

```

Now clean the data by replacing null values

MySQL Workbench interface showing a query window with the following SQL statement:

```

1 • SELECT *
2 FROM mytable
3 WHERE shipped_date IS NULL;
4 • UPDATE mytable
5 SET shipped_date = order_date
6 WHERE shipped_date IS NULL;
7
8
9

```

The Result Grid shows the following data:

order_id	customer_id	employee_id	order_date	required_date	shipped_date	ship_via	freight	ship_name	ship_address
11008	ERNISH	7	0000-00-00	0000-00-00	NULL	3	79.46	Ernst Handel	Kirchgasse 6
11019	RANCH	6	0000-00-00	0000-00-00	NULL	3	3.17	Rancho grande	Av. del Libertador
11039	LINDO	1	0000-00-00	0000-00-00	NULL	2	65.00	LINO-Delicateses	Ave. 5 de Mayo F
11040	GREAL	4	0000-00-00	0000-00-00	NULL	3	18.84	Great Lakes Food Market	2732 Baker Blvd.
11045	BOTTOM	6	0000-00-00	0000-00-00	NULL	2	70.58	Bottom-Dollar Markets	23 Tsawwassen Blv
11051	LAMAI	7	0000-00-00	0000-00-00	NULL	3	2.79	La maison d'Asie	1 rue Alsace-Lorr
11054	CACTU	8	0000-00-00	0000-00-00	NULL	1	0.33	Cactus Comidas para llevar	Cerrito 333
11058	BLAIS	9	0000-00-00	0000-00-00	NULL	3	31.14	Blauer See Delikatessen	Forsterstr. 57
11059	RICAR	2	0000-00-00	0000-00-00	NULL	2	85.80	Ricardo Adocados	Av. Copacabana,
11061	GREAL	4	0000-00-00	0000-00-00	NULL	3	14.01	Great Lakes Food Market	2732 Baker Blvd.
11062	REGGC	4	0000-00-00	0000-00-00	NULL	2	29.93	Reggiani Caseificio	Strada Provinciale
11065	LILAS	8	0000-00-00	0000-00-00	NULL	1	12.91	LILA-Supermercado	Carrera 52 con A
11068	QUEEN	8	0000-00-00	0000-00-00	NULL	2	81.75	Queen Cozinha	Alameda dos Can
11070	LEHMS	2	0000-00-00	0000-00-00	NULL	1	135.00	Lehmanns Marktstand	Magazinweg 7

Get insights

The screenshot shows the MySQL Workbench interface. The left sidebar contains the 'MANAGEMENT' and 'PERFORMANCE' tabs. The 'Administration' tab is selected, showing 'Data Import/Restore' and 'Schemas'. The 'Query Editor' is open, displaying a SQL query: `SELECT ship_country, COUNT(*) AS total_orders FROM mytable GROUP BY ship_country;`. The 'Result Grid' is visible, showing a table with two columns: 'ship_country' and 'total_orders'. The table contains data for various countries, including Argentina, Austria, Belgium, Brazil, Canada, Denmark, Finland, France, Germany, Ireland, Italy, Mexico, Norway, Poland, and the United States. The 'Output' tab at the bottom shows a message: 'Error Code: 1146. Table 'business.mytables' doesn't exist'.

ship_country	total_orders
Argentina	16
Austria	40
Belgium	19
Brazil	83
Canada	30
Denmark	18
Finland	22
France	77
Germany	122
Ireland	19
Italy	28
Mexico	28
Norway	6
Poland	7
United States	10

The queries used are as follows:

SELECT *FROM mytable WHERE shipped_date IS NULL;

UPDATE mytable SET shipped_date = order_date WHERE shipped_date IS NULL;

DELETE FROM mytable WHERE order_date IS NULL;

ALTER TABLE mytable ADD delivery_days INT;

UPDATE mytable SET delivery_days = DATEDIFF(shipped_date, order_date);

ALTER TABLE mytable ADD is_delayed VARCHAR(10);

UPDATE mytable SET is_delayed = CASE WHEN shipped_date > required_date THEN 'Delayed'

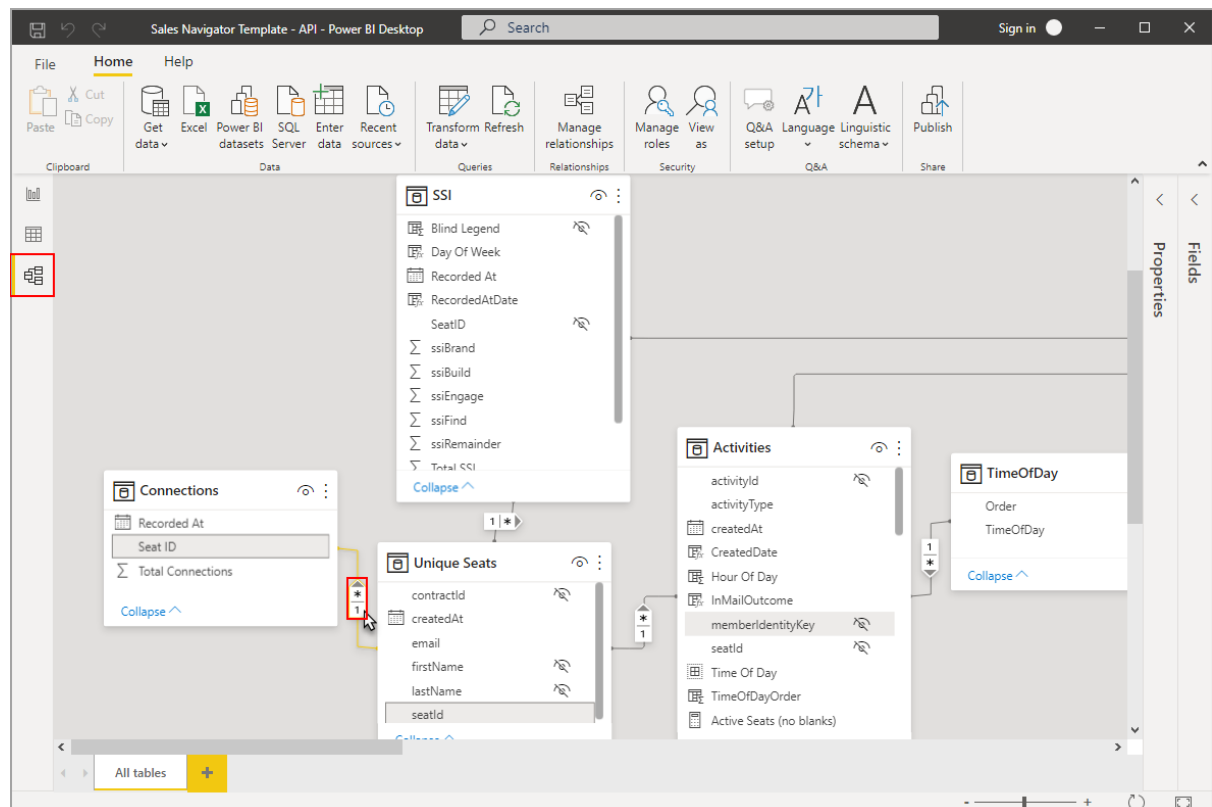
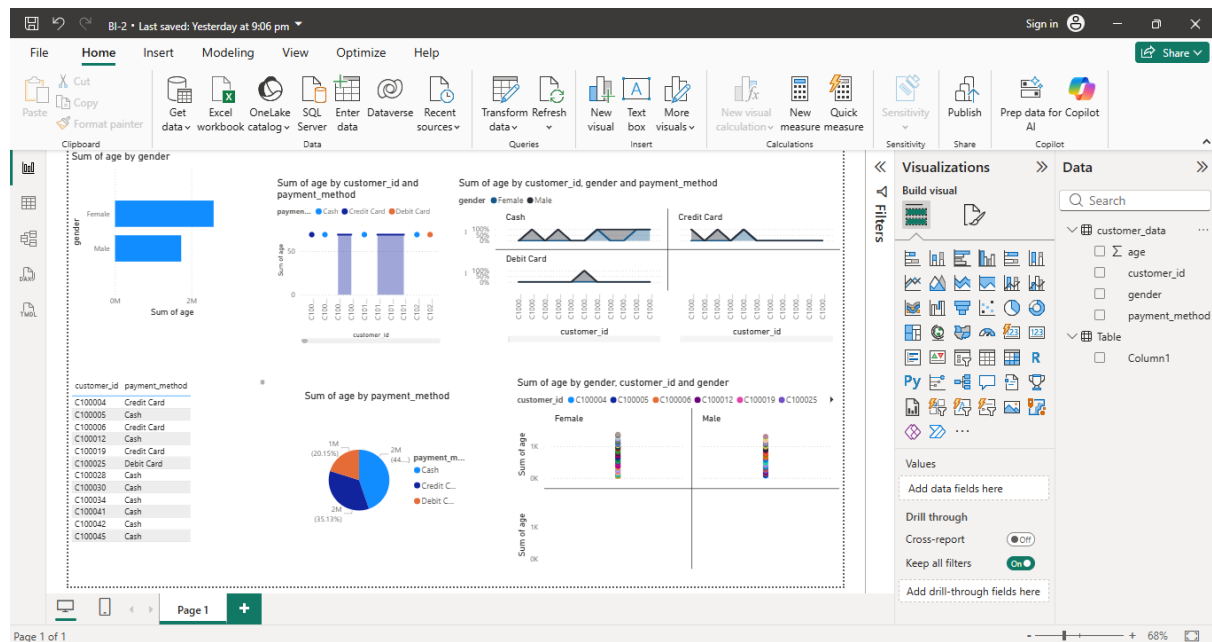
ELSE 'On Time' END;

SELECT ship_country, COUNT(*) AS total_orders FROM mytable GROUP BY ship_country;

SELECT AVG(delivery_days) AS avg_delivery FROM mytable;

SET SQL_SAFE_UPDATES = 0;

Practical 3: Create the cube with suitable dimension and fact tables based on ROLAP, MOLAP and HOLAP model.



Practical 4: Import the data warehouse data in Microsoft Excel and create the Pivot table and Pivot Chart

order_id	customer	employee	order_date	required_date	shipped_date	ship_via	freight	ship_name	ship_address	ship_city	ship_region	ship_post	ship_country
10248	VINET	5	04-07-1996	01-08-1996	16-07-1996	3	32.38	Vins et al	59 rue de Reims			51100	France
10249	TOMSP	6	05-07-1996	16-08-1996	10-07-1996	1	11.61	Toms Spe	Luisenstr.	Münster		44087	Germany
10250	HANAR	4	08-07-1996	05-08-1996				Hanari Car	Rua do Pa	Rio de Jan RJ		05454-876	Brazil
10251	VICTE	3	08-07-1996	05-08-1996				Victuaille	2, rue du	Lyon		69004	France
10252	SUPRD	4	09-07-1996	06-08-1996				Supr	me Boulevard	Charleroi		B-6000	Belgium
10253	HANAR	3	10-07-1996	24-08-1996				Hanari Car	Rua do Pa	Rio de Jan RJ		05454-876	Brazil
10254	CHOPS	5	11-07-1996	08-08-1996				Chop-suei	Hauptstr.	Bern		3012	Switzerland
10255	RICSU	9	12-07-1996	09-08-1996				Richter Su	Starenwe	Genève		1204	Switzerland
10256	WELLI	3	15-07-1996	12-08-1996				Wellington	Rua do	McResende	SP	08737-363	Brazil
10257	HILAA	4	16-07-1996	13-08-1996				HILARION	Carrera 22	San Cristóbal	Táchira	5022	Venezuela
10258	ERNSH	1	17-07-1996	14-08-1996				Ernst Han	Kirchgasse	Graz		8010	Austria
10259	CENTC	4	18-07-1996	15-08-1996				Centro co	Sierras de	México D.F.		5022	Mexico
10260	OTTIK	4	19-07-1996	16-08-1996				Ottilies K	Mehrherrn	Kärlin		50739	Germany
10261	QUEDE	4	19-07-1996	16-08-1996				Que Dela	Rua do Pa	Rio de Jan RJ		02389-673	Brazil
10262	RATTC	8	22-07-1996	19-08-1996				Rattlesna	2817 Milt	Albuquerque	NM	87110	USA
10263	ERNSH	9	23-07-1996	20-08-1996				Ernst Han	Kirchgasse	Graz		8010	Austria
10264	FOLKO	6	24-07-1996	21-08-1996				Folk och f	Åkergat	Bräcke		S-844 67	Sweden
10265	BLONP	2	25-07-1996	22-08-1996				Blondel p	24, place	Strasbourg		67000	France
10266	WARTH	3	26-07-1996	06-09-1996	31-07-1996	3	25.73	Wartian H	Torikatu 3	Oulu		90110	Finland
10267	FRANK	4	29-07-1996	26-08-1996	06-08-1996	1	208.58	Frankenv	Berliner P	München		80805	Germany
10268	GROSR	8	30-07-1996	27-08-1996	02-08-1996	3	66.29	GROSELLA	5A Ave.	Caracas	DF	1081	Venezuela
10269	WHITC	5	31-07-1996	14-08-1996	09-08-1996	1	4.56	White Clo	1029 - 12th	Seattle	WA	98124	USA

Connect to Database Server

Enter the information required to connect to the database server.

1. Server name: localhost

2. Log on credentials:

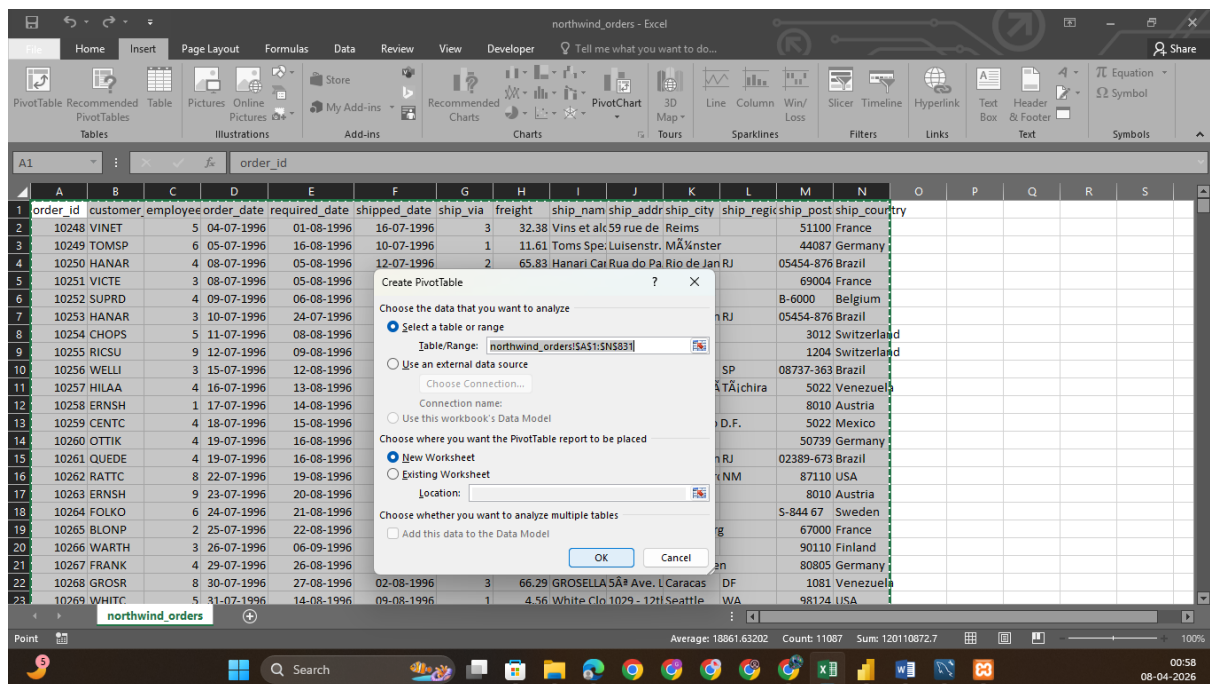
☒ Use Windows Authentication

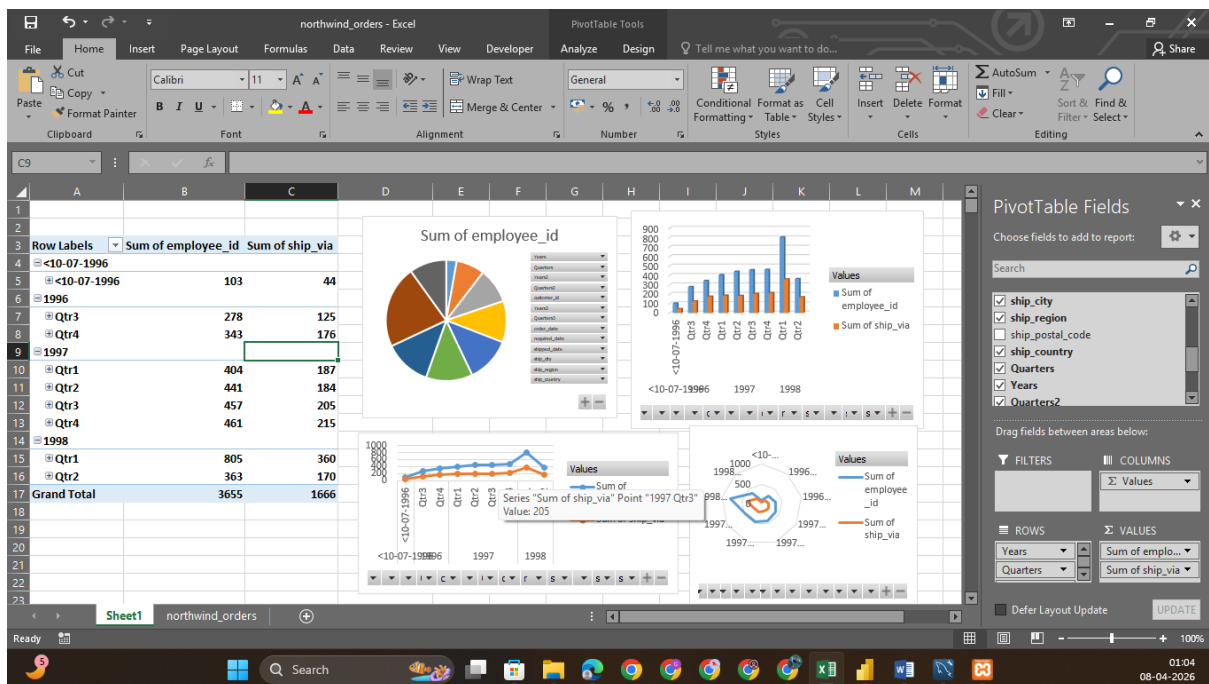
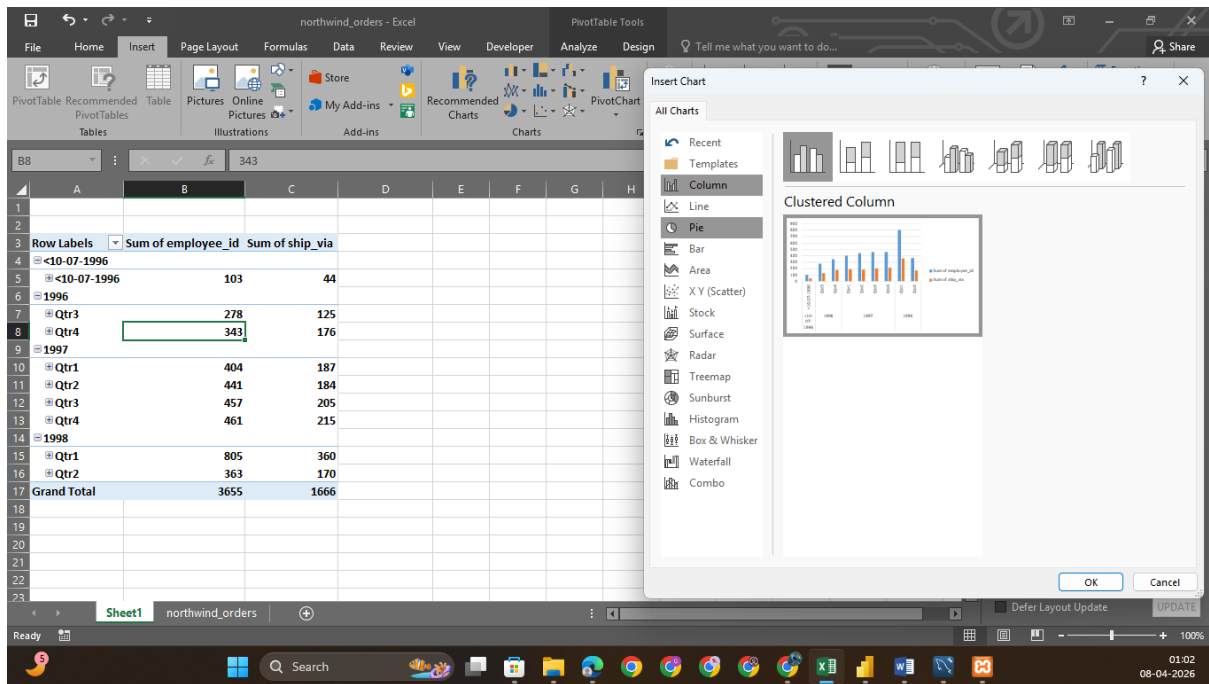
☐ Use the following User Name and Password

User Name:

Password:

Buttons: Cancel, < Back, Next >, Finish





Practical 5: Perform the data classification using classification algorithm. Or
Perform the data clustering using clustering algorithm.

```
[46]: # This Python 3 environment comes with many helpful analytics libraries
      ↪ installed
      # It is defined by the kaggle/python Docker image: https://github.com/kaggle/
      ↪ docker-python
      # For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list
      ↪ all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that
      ↪ gets preserved as output when you create a version using "Save & Run All"
# You can also write temporary files to /kaggle/temp/, but they won't be saved
      ↪ outside of the current session
```

/kaggle/input/sample-sales-data/sales_data_sample.csv

```
[47]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=UserWarning)
```

```
df = pd.read_csv("/kaggle/input/sample-sales-data/sales_data_sample.csv",
encoding='latin1')
```

```
[48]: df.head()
```

```
[48]:
```

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	\
0	10107	30	95.70	2	2871.00	
1	10121	34	81.35	5	2765.90	
2	10134	41	94.74	2	3884.34	
3	10145	45	83.26	6	3746.70	
4	10159	49	100.00	14	5205.27	

	ORDERDATE	STATUS	QTR_ID	MONTH_ID	YEAR_ID	...	\
0	2/24/2003 0:00	Shipped	1	2	2003	...	
1	5/7/2003 0:00	Shipped	2	5	2003	...	
2	7/1/2003 0:00	Shipped	3	7	2003	...	
3	8/25/2003 0:00	Shipped	3	8	2003	...	
4	10/10/2003 0:00	Shipped	4	10	2003	...	

	ADDRESSLINE1	ADDRESSLINE2	CITY	STATE	\
0	897 Long Airport Avenue	NaN	NYC	NY	
1	59 rue de l'Abbaye	NaN	Reims	NaN	
2	27 rue du Colonel Pierre Avia	NaN	Paris	NaN	
3	78934 Hillside Dr.	NaN	Pasadena	CA	
4	7734 Strong St.	NaN	San Francisco	CA	

	POSTALCODE	COUNTRY	TERRITORY	CONTACTLASTNAME	CONTACTFIRSTNAME	DEALSIZE
0	10022	USA	NaN	Yu	Kwai	Small
1	51100	France	EMEA	Henriot	Paul	Small
2	75508	France	EMEA	Da Cunha	Daniel	Medium
3	90003	USA	NaN	Young	Julie	Medium
4	NaN	USA	NaN	Brown	Julie	Medium

[5 rows x 25 columns]

```
[49]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2823 entries, 0 to 2822
Data columns (total 25 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ORDERNUMBER           2823 non-null  int64
1   QUANTITYORDERED       2823 non-null  int64
2   PRICEEACH             2823 non-null  float64
3   ORDERLINENUMBER       2823 non-null  int64
4   SALES                 2823 non-null  float64
```

5	ORDERDATE	2823	non-null	object
6	STATUS	2823	non-null	object
7	QTR_ID	2823	non-null	int64
8	MONTH_ID	2823	non-null	int64
9	YEAR_ID	2823	non-null	int64
10	PRODUCTLINE	2823	non-null	object
11	MSRP	2823	non-null	int64
12	PRODUCTCODE	2823	non-null	object
13	CUSTOMERNAME	2823	non-null	object
14	PHONE	2823	non-null	object
15	ADDRESSLINE1	2823	non-null	object
16	ADDRESSLINE2	302	non-null	object
17	CITY	2823	non-null	object
18	STATE	1337	non-null	object
19	POSTALCODE	2747	non-null	object
20	COUNTRY	2823	non-null	object
21	TERRITORY	1749	non-null	object
22	CONTACTLASTNAME	2823	non-null	object
23	CONTACTFIRSTNAME	2823	non-null	object
24	DEALSIZE	2823	non-null	object

dtypes: float64(2), int64(7), object(16)

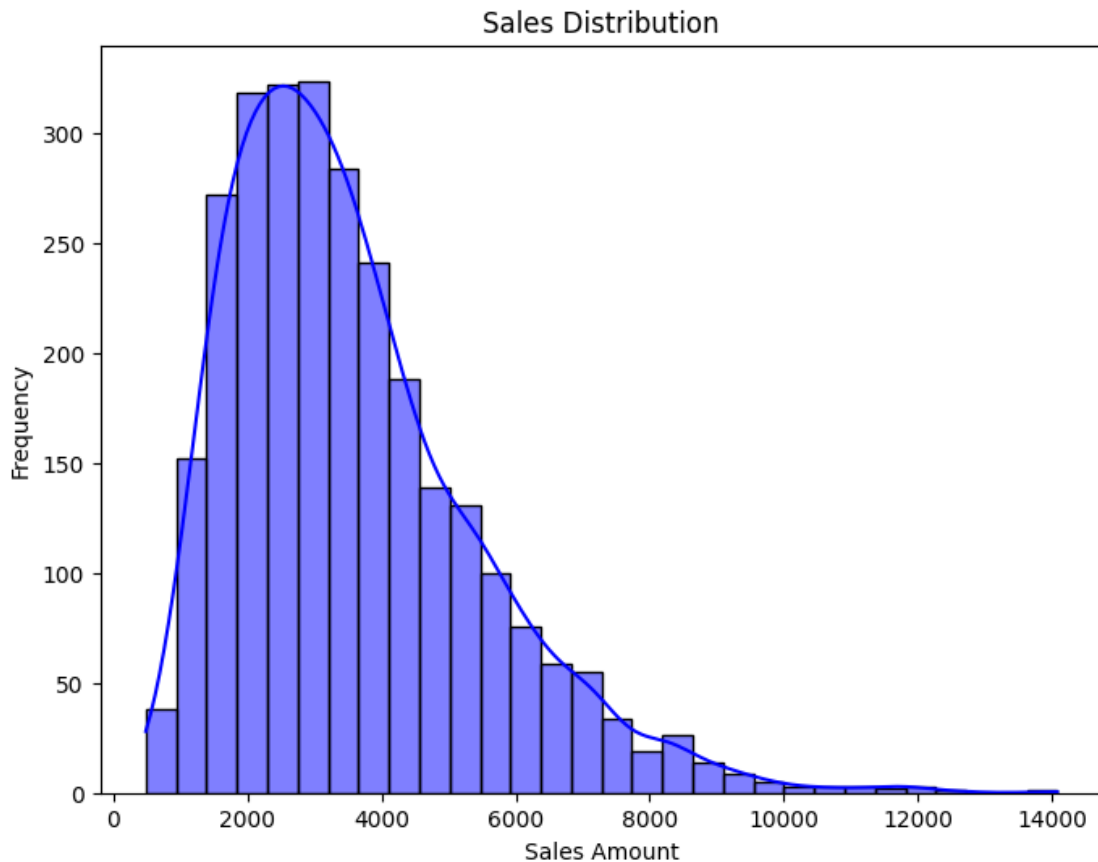
memory usage: 551.5+ KB

```
[50]: df.isnull().sum()
```

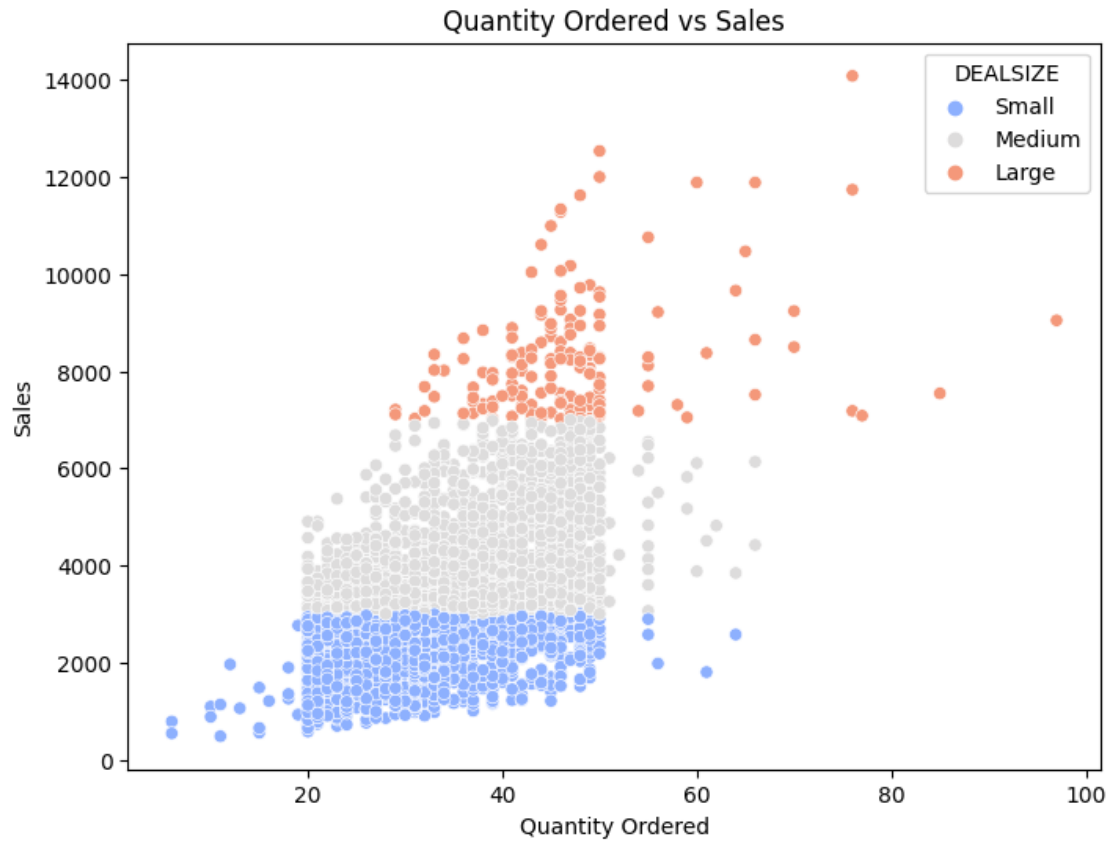
```
[50]: ORDERNUMBER      0
      QUANTITYORDERED  0
      PRICEEACH        0
      ORDERLINENUMBER  0
      SALES            0
      ORDERDATE        0
      STATUS          0
      QTR_ID          0
      MONTH_ID        0
      YEAR_ID         0
      PRODUCTLINE     0
      MSRP            0
      PRODUCTCODE     0
      CUSTOMERNAME    0
      PHONE           0
      ADDRESSLINE1     0
      ADDRESSLINE2    2521
      CITY            0
      STATE           1486
      POSTALCODE       76
      COUNTRY         0
      TERRITORY       1074
```

```
CONTACTLASTNAME    0
CONTACTFIRSTNAME   0
DEALSIZE            0
dtype: int64
```

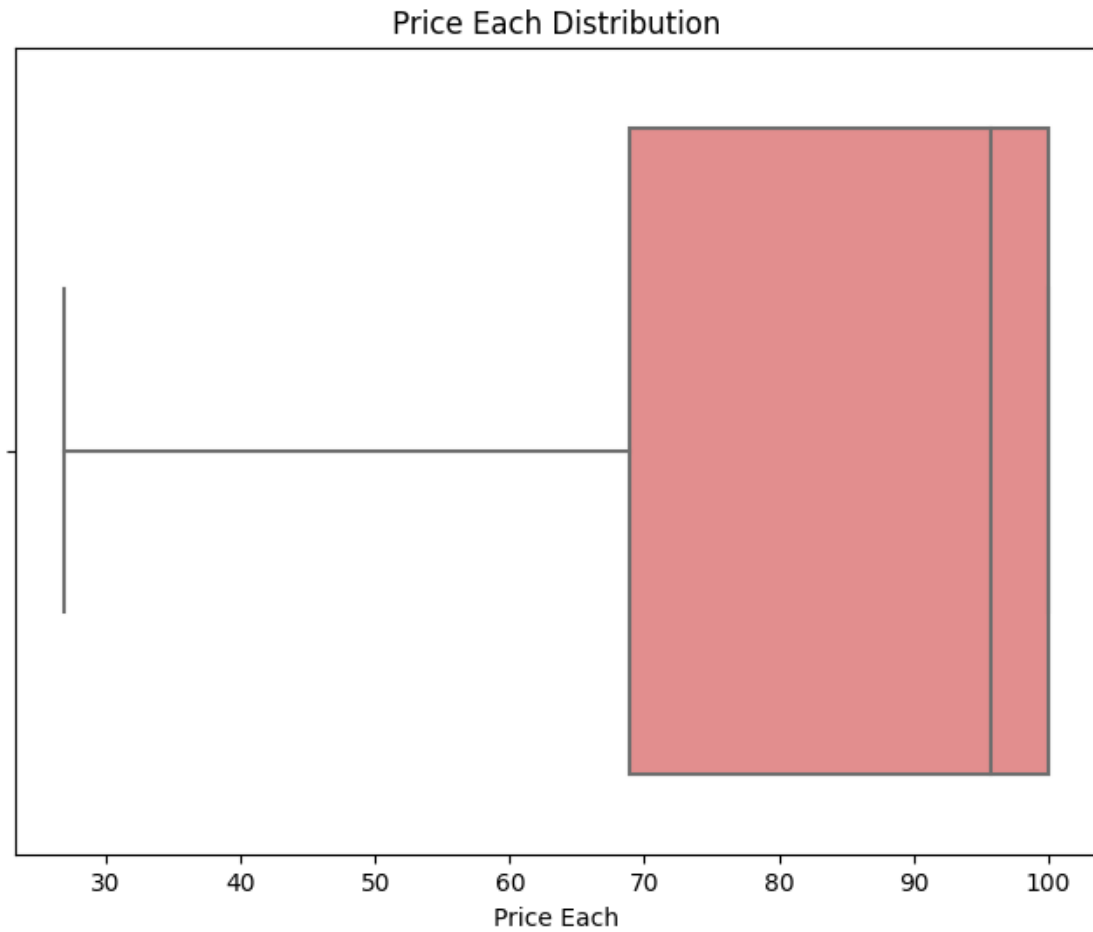
```
[51]: plt.figure(figsize=(8,6))
sns.histplot(df['SALES'], kde=True, color='blue', bins=30)
plt.title('Sales Distribution')
plt.xlabel('Sales Amount')
plt.ylabel('Frequency')
plt.show()
```



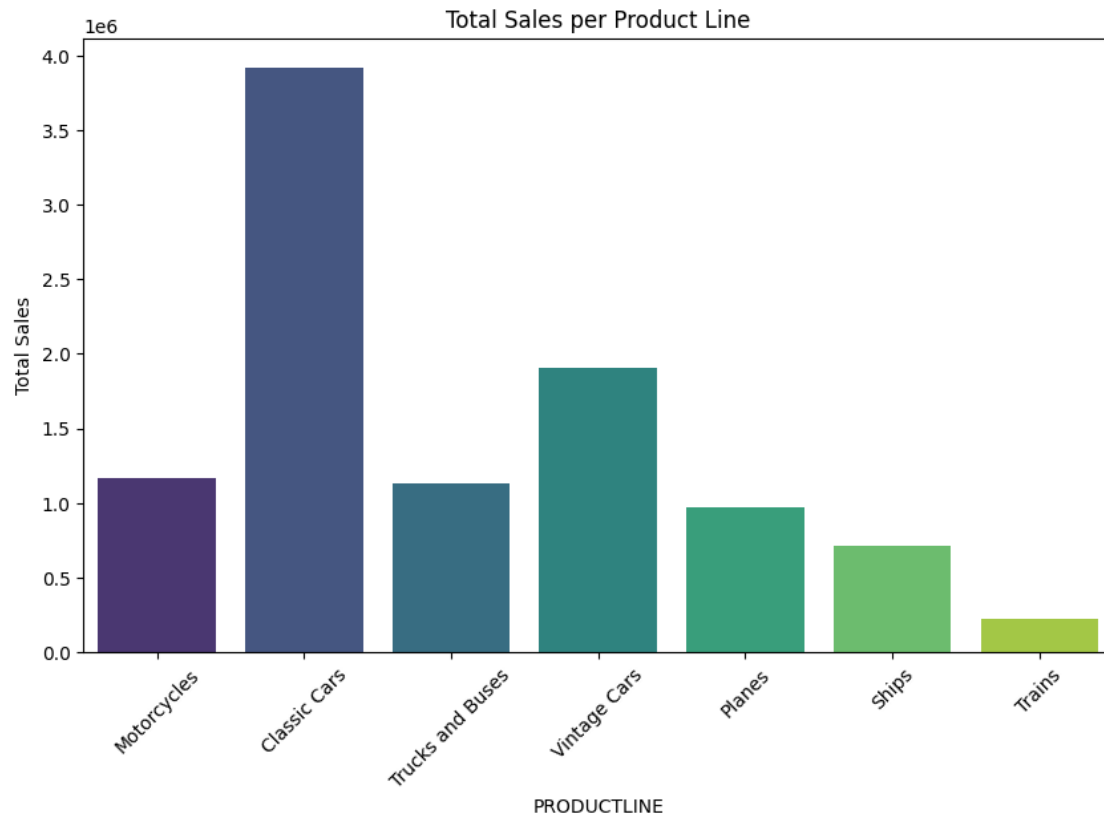
```
[52]: plt.figure(figsize=(8,6))
sns.scatterplot(x='QUANTITYORDERED', y='SALES', data=df, hue='DEALSIZE',
               palette='coolwarm')
plt.title('Quantity Ordered vs Sales')
plt.xlabel('Quantity Ordered')
plt.ylabel('Sales')
plt.show()
```



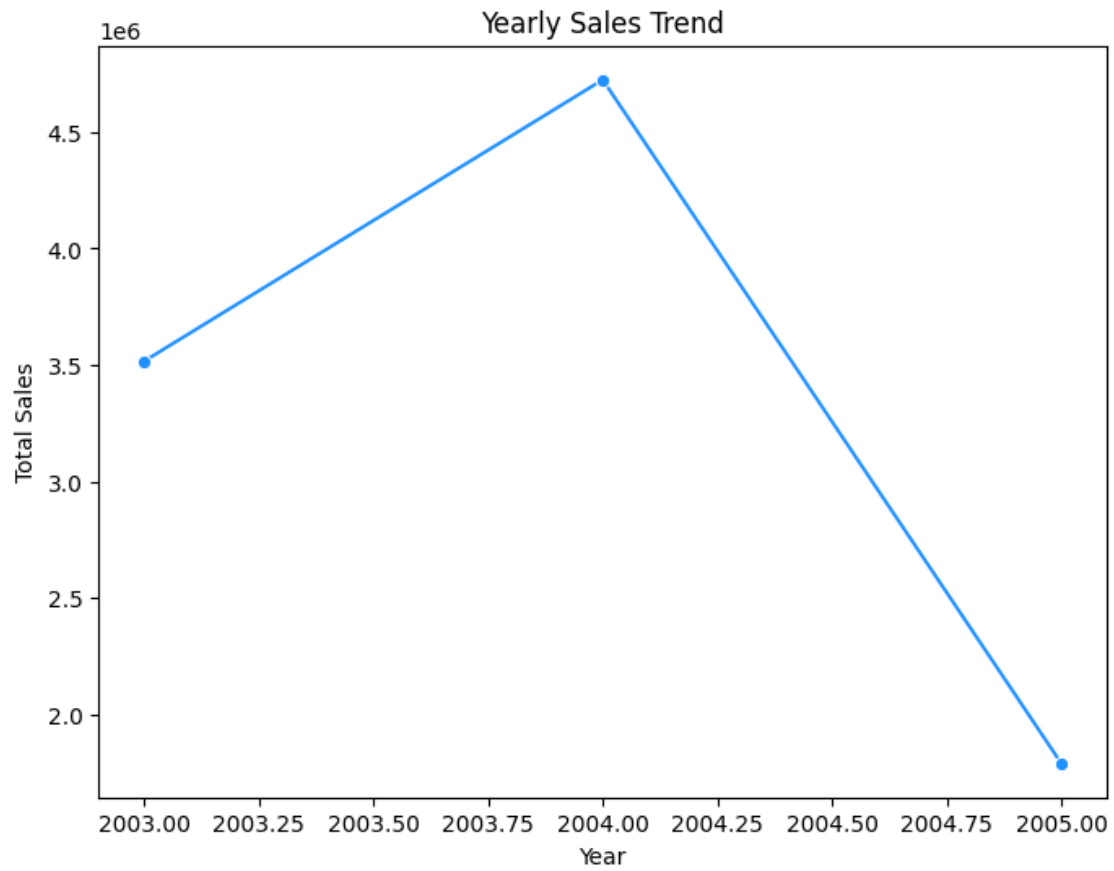
```
[53]: plt.figure(figsize=(8,6))
sns.boxplot(x='PRICEEACH', data=df, color='lightcoral')
plt.title('Price Each Distribution')
plt.xlabel('Price Each')
plt.show()
```

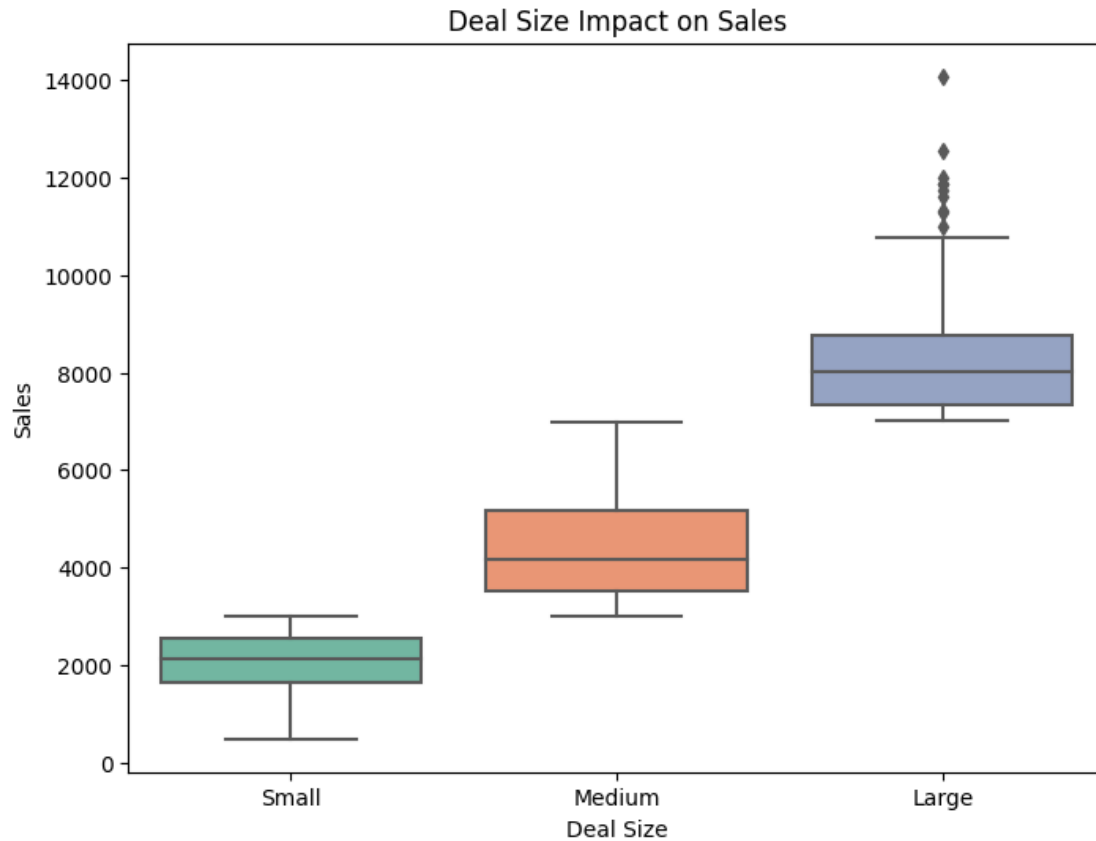
```
[54]: plt.figure(figsize=(10,6))
sns.barplot(x='PRODUCTLINE', y='SALES', data=df, estimator=sum, ci=None,
           palette='viridis')
plt.title('Total Sales per Product Line')
plt.xticks(rotation=45)
plt.ylabel('Total Sales')
plt.show()
```



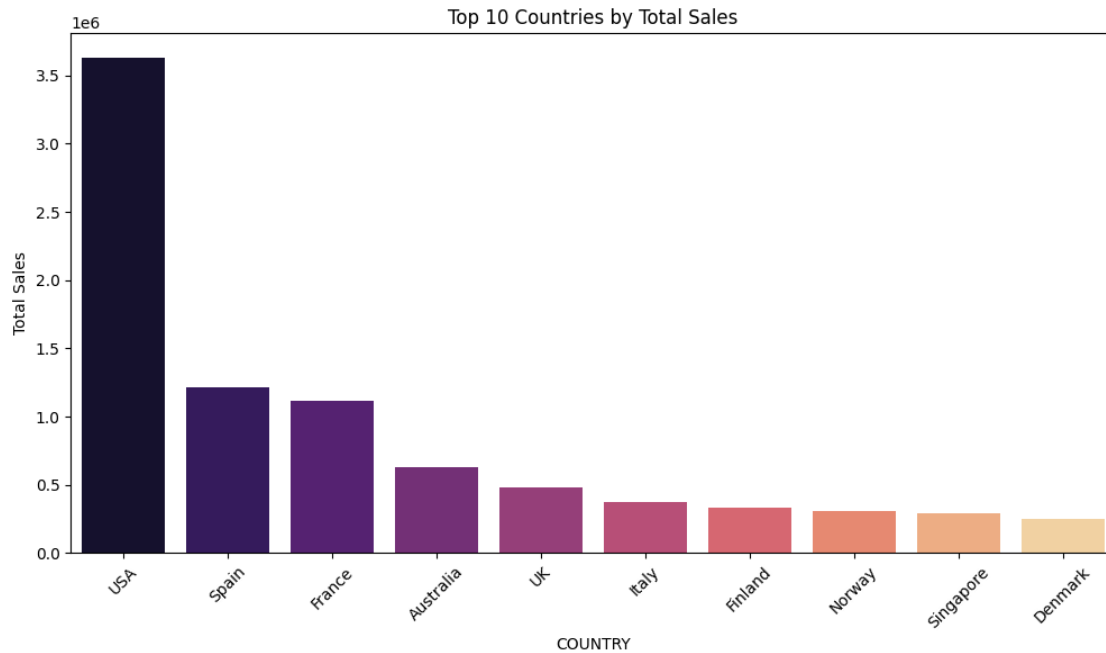
```
[55]: plt.figure(figsize=(8,6))
sales_per_year = df.groupby('YEAR_ID')['SALES'].sum().reset_index()
sns.lineplot(x='YEAR_ID', y='SALES', data=sales_per_year, marker='o',
             color='dodgerblue')
plt.title('Yearly Sales Trend')
plt.xlabel('Year')
plt.ylabel('Total Sales')
plt.show()
```



```
[56]: plt.figure(figsize=(8,6))
sns.boxplot(x='DEALSIZE', y='SALES', data=df, palette='Set2')
plt.title('Deal Size Impact on Sales')
plt.xlabel('Deal Size')
plt.ylabel('Sales')
plt.show()
```



```
[57]: plt.figure(figsize=(12,6))
top_countries = df.groupby('COUNTRY')['SALES'].sum().nlargest(10).reset_index()
sns.barplot(x='COUNTRY', y='SALES', data=top_countries, palette='magma')
plt.title('Top 10 Countries by Total Sales')
plt.xticks(rotation=45)
plt.ylabel('Total Sales')
plt.show()
```



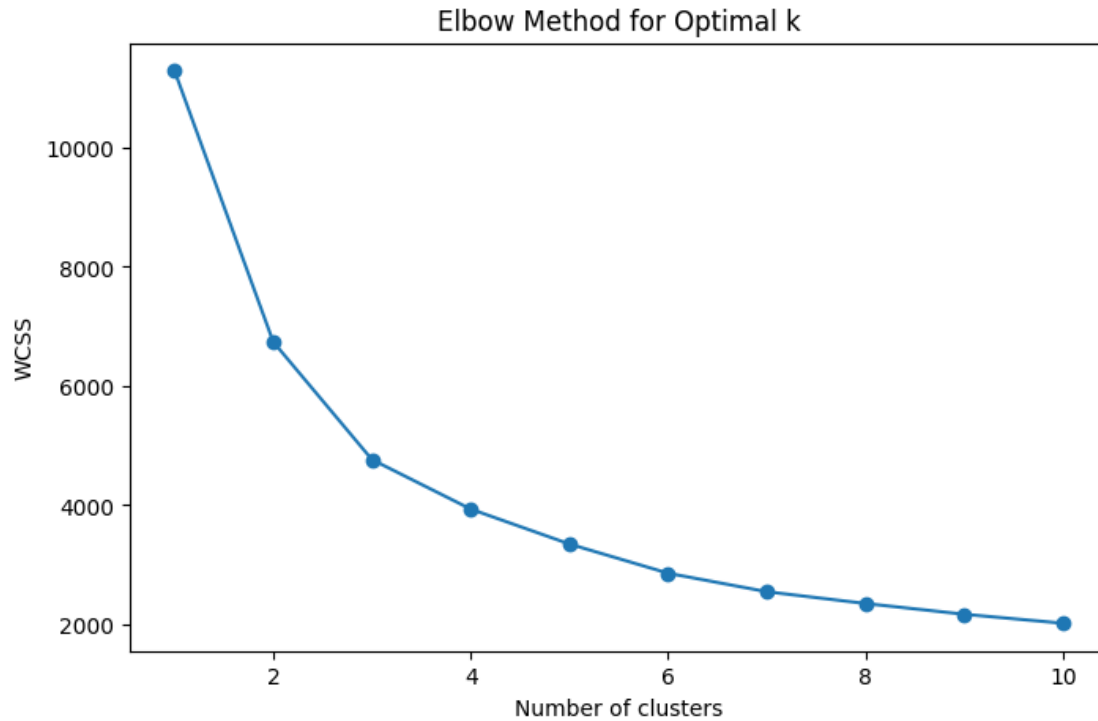
```
[58]: # Select numerical features for clustering
features = ['QUANTITYORDERED', 'PRICEEACH', 'SALES', 'MSRP']
X = df[features]
```

```
[59]: # Handle missing values (if any)
X = X.dropna()

# Standardize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
[60]: # Calculate WCSS for different cluster sizes
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

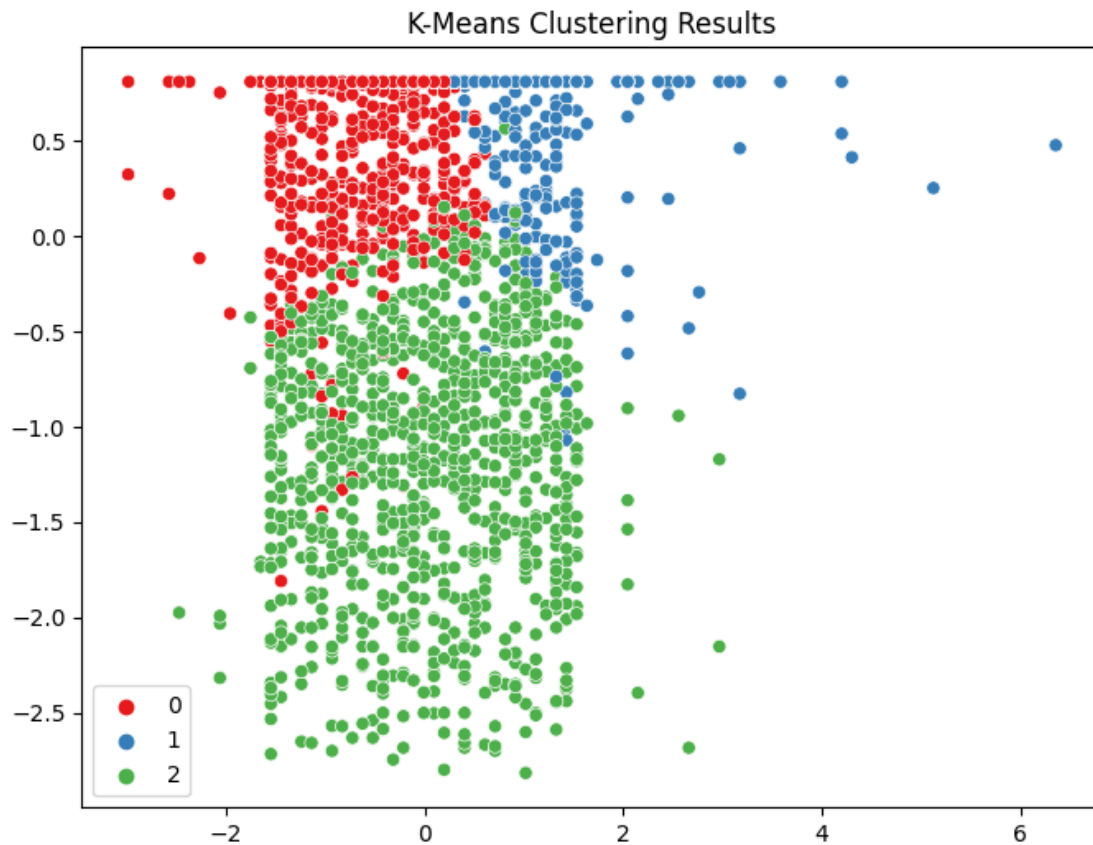
# Plot the Elbow Curve
plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.show()
```



```
[61]: # Choose k based on the elbow method, e.g., k=3
kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(X_scaled)

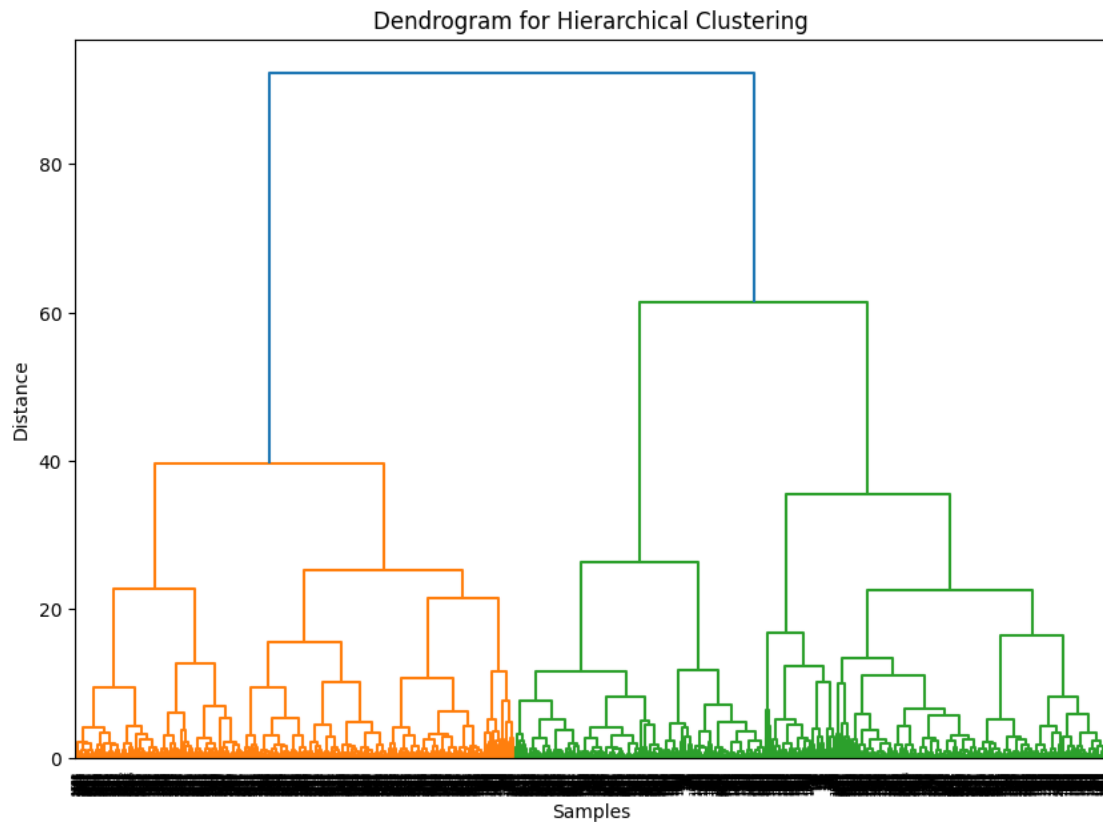
# Add the cluster labels to the original dataframe
df['Cluster'] = clusters

# Visualize the clusters
plt.figure(figsize=(8, 6))
sns.scatterplot(x=X_scaled[:, 0], y=X_scaled[:, 1], hue=clusters,
               palette='Set1')
plt.title('K-Means Clustering Results')
plt.show()
```



```
[62]: # Generate linkage matrix using the 'ward' method
linked = linkage(X_scaled, method='ward')

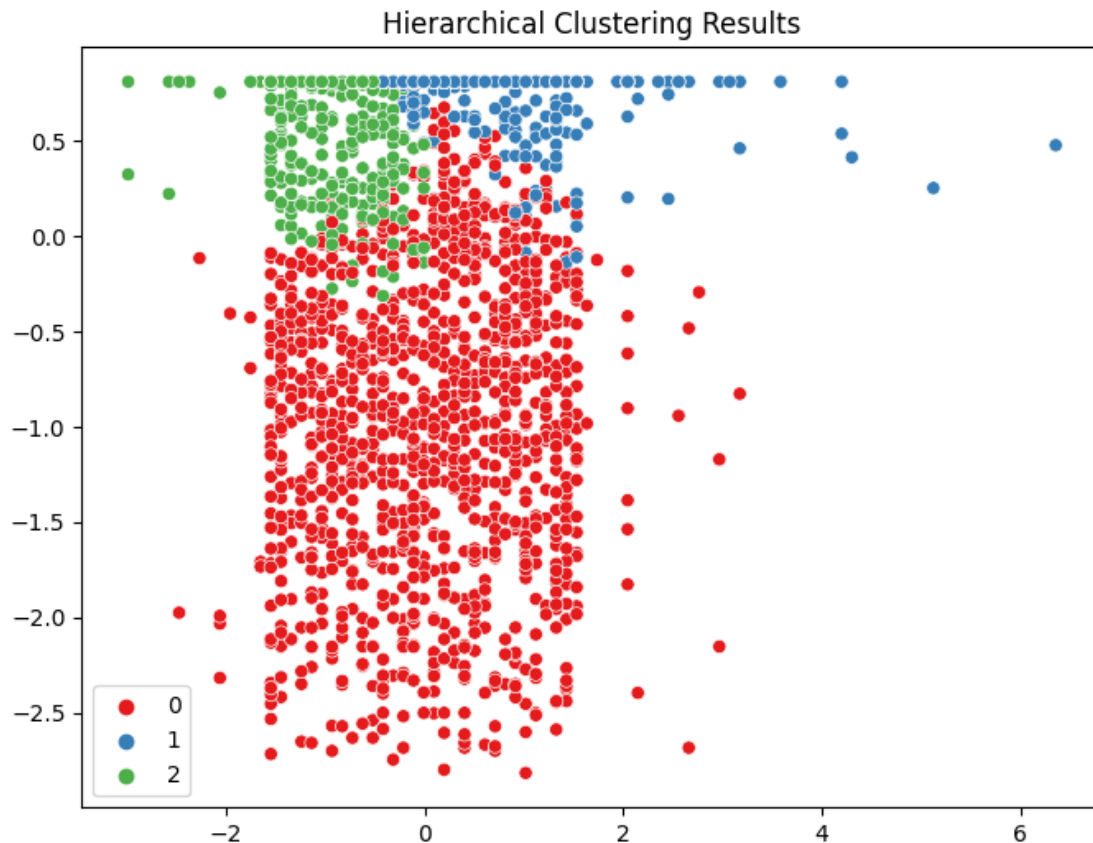
# Plot the dendrogram
plt.figure(figsize=(10, 7))
dendrogram(linked)
plt.title('Dendrogram for Hierarchical Clustering')
plt.xlabel('Samples')
plt.ylabel('Distance')
plt.show()
```



```
[63]: # Choose the number of clusters, e.g., 3
hierarchical_clustering = AgglomerativeClustering(n_clusters=3,
    ↪affinity='euclidean', linkage='ward')
hc_clusters = hierarchical_clustering.fit_predict(X_scaled)

# Add the cluster labels to the original dataframe
df['HC_Cluster'] = hc_clusters

# Visualize the clusters
plt.figure(figsize=(8, 6))
sns.scatterplot(x=X_scaled[:, 0], y=X_scaled[:, 1], hue=hc_clusters,
    ↪palette='Set1')
plt.title('Hierarchical Clustering Results')
plt.show()
```

```
[64]: # Inertia for K-Means
inertia = kmeans.inertia_
print(f'Inertia: {inertia}')

from sklearn.metrics import silhouette_score
# Silhouette Score for K-Means
kmeans_silhouette = silhouette_score(X_scaled, clusters)
print(f'Silhouette Score (K-Means): {kmeans_silhouette}')
# Silhouette Score for Hierarchical Clustering
hc_silhouette = silhouette_score(X_scaled, hc_clusters)
print(f'Silhouette Score (Hierarchical): {hc_silhouette}')

from sklearn.metrics import davies_bouldin_score
# Davies-Bouldin Index for K-Means
kmeans_dbi = davies_bouldin_score(X_scaled, clusters)
print(f'Davies-Bouldin Index (K-Means): {kmeans_dbi}')
# Davies-Bouldin Index for Hierarchical Clustering
hc_dbi = davies_bouldin_score(X_scaled, hc_clusters)
print(f'Davies-Bouldin Index (Hierarchical): {hc_dbi}')
```

```
from sklearn.metrics import calinski_harabasz_score
# Calinski-Harabasz Index for K-Means
kmeans_ch = calinski_harabasz_score(X_scaled, clusters)
print(f'Calinski-Harabasz Index (K-Means): {kmeans_ch}')
# Calinski-Harabasz Index for Hierarchical Clustering
hc_ch = calinski_harabasz_score(X_scaled, hc_clusters)
print(f'Calinski-Harabasz Index (Hierarchical): {hc_ch}')
```

Inertia: 4766.017133863959

Silhouette Score (K-Means): 0.3504523735351092

Silhouette Score (Hierarchical): 0.31527494740967016

Davies-Bouldin Index (K-Means): 1.0091895570978797

Davies-Bouldin Index (Hierarchical): 0.9888199869306714

Calinski-Harabasz Index (K-Means): 1930.6761983442068

Calinski-Harabasz Index (Hierarchical): 1672.3553032169289